Deep Learning (Spring 2026)
Solution for Assignment 6

Pan Changxun
2024011323

April 23, 2026

Solution 1.

1. **Most computationally expensive block.**

   A vanilla transformer layer is composed of a multi-head self-attention (MHSA) sub-layer and a position-wise feed-forward (FFN) sub-layer. For a sequence of length L and model dimension d (with FFN hidden dimension $d_{\text{ff}} = 4d$), the dominant FLOP counts per layer are:

$$\begin{aligned} \text{MHSA}: &\quad \underbrace{4\,L\,d^2}_{\text{Q, K, V, O projections}} + \underbrace{2\,L^2\,d}_{\text{attention scores and aggregation}}, \\ \text{FFN}: &\quad \underbrace{2\,L\,d\,d_{\text{ff}}}_{\text{two linear maps}} = 8\,L\,d^2. \end{aligned}$$

   When the sequence is long enough that $L \gg d$, the quadratic term $2L^2d$ in self-attention dominates. This is precisely the regime that causes the training-memory blow-up mentioned in the question: the $L \times L$ attention matrix has to be materialized for both the forward pass and the backward pass (to compute gradients through softmax), so GPU memory grows as $\Theta(L^2)$ per head per layer. Therefore the **multi-head self-attention block is the most computationally expensive part of a vanilla transformer**, because its cost and memory footprint scale *quadratically* in the sequence length, whereas the FFN scales only linearly.

2. **Truncated-BPTT analogue for transformers.**

   Truncated BPTT for RNNs splits a long sequence into fixed-length segments, processes them sequentially, carries the last hidden state across segments as a "context," and *stops gradients* from flowing back through that context. We can do the same thing for transformers by adapting the Transformer-XL recurrence mechanism of Dai et al. (2019).

   Key idea. Partition the long input $x_{1:N}$ into T consecutive segments $s_1, s_2, \ldots, s_T$, each of length L (with $N = TL$). For every layer ℓ we maintain a *memory cache* $M^{(\ell)}$ that stores the hidden states produced for the previous segment(s) at layer ℓ. When processing the current segment we let each layer attend over the concatenation of its cache and the current-segment activations, but we *detach* the cache before using it so no gradient flows back into earlier segments. Because segment boundaries change absolute positions, we use *relative* positional encoding so that attention scores only depend on token offsets. This gives transformers the same linear-in-N memory profile that truncated BPTT gives RNNs, at the cost of a bias of $O(LM)$ steps beyond which gradients are truncated (here $M = |M^{(\ell)}|$ is the cache length).

   Pseudo-code. Let D be the number of transformer layers, L the segment length, and M the memory length (typically $M \in \{L, 2L\}$).

Algorithm: Truncated BPTT for a Transformer (Transformer-XL style).

Input: long sequence $x_{1:N}$; segment length L; memory length M; number of layers D; parameters θ.
Output: trained parameters θ.

(a) Partition $x_{1:N}$ into segments $s_1, s_2, \ldots, s_T$ of length L.

(b) Initialize $\mathrm{Mem}^{(\ell)} \leftarrow \emptyset$ for $\ell = 0, 1, \ldots, D-1$. *(empty memory per layer)*

(c) **For** $t = 1, 2, \ldots, T$ **do** *(iterate over segments)*

  i. $h^{(0)} \leftarrow \mathrm{Embed}(s_t)$ *(shape $L \times d$)*

  ii. **For** $\ell = 1, 2, \ldots, D$ **do**

    A. $\tilde{h}^{(\ell-1)} \leftarrow \mathrm{concat}\big(\mathrm{stop_grad}(\mathrm{Mem}^{(\ell-1)}),\ h^{(\ell-1)}\big)$
    (detach memory – this is where BPTT is truncated)

    B. $h^{(\ell)} \leftarrow \mathrm{TransformerLayer}_\ell\big(Q = h^{(\ell-1)},\ K = V = \tilde{h}^{(\ell-1)};\ \text{relative pos. enc.}\big)$

  iii. $\mathcal{L}_t \leftarrow \mathrm{CrossEntropy}\big(\mathrm{Out}(h^{(D)}),\ \text{targets of } s_t\big)$.

  iv. Backpropagate $\mathcal{L}_t$ and update θ. *(gradient does not cross* stop_grad*)*

  v. **For** $\ell = 0, 1, \ldots, D-1$ **do** *(refresh memory)*

    A. $\mathrm{Mem}^{(\ell)} \leftarrow \mathrm{stop_grad}\big(\text{last } M \text{ rows of } \mathrm{concat}(\mathrm{Mem}^{(\ell)},\ h^{(\ell)})\big)$

Why this works.

- *Memory is bounded.* At any moment we only store $O\big(D \cdot (L+M) \cdot d\big)$ activations, instead of $O(D \cdot N \cdot d)$ for the naive case.
- *Gradients are truncated.* Because $M^{(\ell)}$ is detached, back-prop at step t only traverses the current segment, analogous to truncated BPTT for RNNs.
- *Context is preserved.* Forward attention still reaches M tokens into the past at every layer; stacking D layers effectively extends the receptive field by $D \cdot M$ tokens.
- *Positions remain consistent.* Using relative positional encoding is essential – with the vanilla absolute encoding, tokens in $M^{(\ell)}$ and in s_t would carry overlapping position embeddings across segments.

Solution 2. We construct the map $z \mapsto x$ as an L-fold composition of elementary affine-coupling layers, each of which alters exactly one coordinate.

Layer definition. Let $w^{(0)} \triangleq z \in \mathbb{R}^L$ and write $w^{(i)}_{<i} = (w^{(i)}_1, \ldots, w^{(i)}_{i-1})$. Define the i-th layer $T_i : \mathbb{R}^L \to \mathbb{R}^L$, for $i = 1, 2, \ldots, L$, by

$$\big(T_i(w)\big)_j = \begin{cases} w_j, & j \neq i, \\ \mu_\theta\big(w_1, \ldots, w_{i-1}\big) \ + \ w_i \cdot \exp\big(\alpha_\theta\big(w_1, \ldots, w_{i-1}\big)\big), & j = i, \end{cases} \tag{1}$$

and set $w^{(i)} = T_i(w^{(i-1)})$, so that the full flow is

$$x \ = \ T(z) \ = \ \big(T_L \circ T_{L-1} \circ \cdots \circ T_1\big)(z).$$

At the input to T_i we have $w^{(i-1)}_j = x_j$ for $j < i$ and $w^{(i-1)}_j = z_j$ for $j \geq i$ (earlier layers have only rewritten coordinates $1, \ldots, i-1$). Since the map $(z_1, \ldots, z_{i-1}) \leftrightarrow (x_1, \ldots, x_{i-1})$ produced by the earlier layers is a bijection, conditioning on $w^{(i-1)}_{<i} = x_{<i}$ is equivalent to conditioning on $z_{<i}$; in other words the Parallel WaveNet formula $x_i = \mu_\theta(z_{1:i-1}) + z_i \exp(\alpha_\theta(z_{1:i-1}))$ is reproduced exactly by (1).

Invertibility of each layer. Because w_j is unchanged for $j \neq i$ and $\exp(\alpha_\theta(\cdot)) > 0$ everywhere, T_i admits a closed-form inverse:

$$\big(T_i^{-1}(w')\big)_i \;=\; \frac{w'_i \;-\; \mu_\theta(w'_{<i})}{\exp\big(\alpha_\theta(w'_{<i})\big)}, \qquad \big(T_i^{-1}(w')\big)_j = w'_j \;\; (j \neq i).$$

Composition of bijections is a bijection, so $T = T_L \circ \cdots \circ T_1$ is invertible.

Jacobian of each layer. The Jacobian $J_i \triangleq \partial T_i(w)/\partial w$ has unit rows for every $j \neq i$ (since those coordinates are copied through) and a single non-trivial row at index i:

$$\big(J_i\big)_{ij} \;=\; \begin{cases} \exp\big(\alpha_\theta(w_{<i})\big), & j = i, \\ \frac{\partial \mu_\theta(w_{<i})}{\partial w_j} \;+\; w_i \, \exp\big(\alpha_\theta(w_{<i})\big) \, \frac{\partial \alpha_\theta(w_{<i})}{\partial w_j}, & j < i, \\ 0, & j > i. \end{cases}$$

Therefore J_i is *lower triangular* with diagonal

$$\mathrm{diag}(J_i) \;=\; \big(1,\; \ldots,\; 1,\; \underbrace{\exp\big(\alpha_\theta(w_{<i})\big)}_{\text{position } i},\; 1,\; \ldots,\; 1\big),$$

and its determinant is simply

$$\det(J_i) \;=\; \exp\big(\alpha_\theta\big(w_{<i}^{(i-1)}\big)\big) \;=\; \exp\big(\alpha_\theta(z_{1:i-1})\big). \tag{2}$$

Why the composition is a valid flow. A valid normalizing flow is (i) a diffeomorphism whose (ii) log-determinant is tractable. We have shown (i) above. For (ii), the Jacobian of the composition factorizes via the chain rule:

$$\frac{\partial x}{\partial z} \;=\; J_L \cdot J_{L-1} \cdots J_1, \qquad \log\big|\det(\partial x/\partial z)\big| \;=\; \sum_{i=1}^{L} \log\big|\det(J_i)\big| \;=\; \sum_{i=1}^{L} \alpha_\theta(z_{1:i-1}),$$

using (2). Both terms can be computed in one pass over the L coordinates, so the flow is efficient. By the change-of-variables formula,

$$\log p_X(x) \;=\; \log p_Z(z) \;-\; \sum_{i=1}^{L} \alpha_\theta(z_{1:i-1}) \;=\; -\frac{1}{2}\sum_{i=1}^{L} z_i^2 \;-\; \frac{L}{2}\log(2\pi) \;-\; \sum_{i=1}^{L} \alpha_\theta(z_{1:i-1}),$$

which is the tractable log-likelihood of the Parallel WaveNet flow. Hence the L-layer construction (1) yields a valid normalizing flow that realizes the generative process of Parallel WaveNet. ■

Solution 3. Fix an input x. We prove the result in two steps: a single-token lemma, then an induction on sequence length.

Setup. At a given position, write $p(y) \triangleq p(y \mid x, y_<)$ for the target's next-token distribution and $q(y) \triangleq q(y \mid x, y_<)$ for the draft's, where $y_<$ is the already-generated prefix. Speculative decoding uses the following single-step acceptance rule (Leviathan et al. 2023):

(a) Draw a candidate $\tilde{y} \sim q$.

(b) Draw $r \sim \mathrm{Uniform}(0, 1)$. If $r \leq \min\big(1,\, p(\tilde{y})/q(\tilde{y})\big)$, *accept* and output $\tilde{y}$.

(c) Otherwise *reject* and output a fresh sample $y^\star$ drawn from the *residual distribution*

$$p_{\mathrm{res}}(y) \;\triangleq\; \frac{\big(p(y) - q(y)\big)_+}{Z}, \qquad Z \;=\; \sum_{y'} \big(p(y') - q(y')\big)_+, \tag{3}$$

where $(a)_+ \triangleq \max(a, 0)$.

Lemma (single-token correctness). *Under the rule above, the output token Y satisfies* $\Pr[Y = y] = p(y)$ *for every* y.

Proof. A useful identity, provable by a case split on the sign of $p - q$, is

$$\min(p(y), q(y)) \;+\; \big(p(y) - q(y)\big)_+ \;=\; p(y) \qquad \text{for every } y. \tag{4}$$

Summing (4) over y gives

$$Z \;=\; \sum_y \big(p(y) - q(y)\big)_+ \;=\; 1 \;-\; \sum_y \min(p(y), q(y)). \tag{5}$$

The probability that a candidate is accepted is

$$P_{\text{acc}} \;=\; \sum_y q(y) \cdot \min\big(1,\, p(y)/q(y)\big) \;=\; \sum_y \min\big(q(y),\, p(y)\big),$$

so the probability of rejection is exactly

$$P_{\text{rej}} \;=\; 1 - P_{\text{acc}} \;=\; 1 - \sum_y \min(p(y), q(y)) \;\overset{(5)}{=}\; Z. \tag{6}$$

Fix any y. There are two disjoint ways to output y:

- *Accepted path:* the draft proposed y and it was accepted. Its probability is

$$q(y) \cdot \min\big(1,\, p(y)/q(y)\big) \;=\; \min\big(q(y), p(y)\big).$$

- *Rejected path: some* draft token was rejected, and the resample landed on y. Its probability is

$$P_{\text{rej}} \cdot p_{\text{res}}(y) \;=\; Z \cdot \frac{\big(p(y) - q(y)\big)_+}{Z} \;=\; \big(p(y) - q(y)\big)_+,$$

  where we used (6) in the first step.

Adding the two contributions and applying (4),

$$\Pr[Y = y] \;=\; \min\big(q(y), p(y)\big) \;+\; \big(p(y) - q(y)\big)_+ \;=\; p(y),$$

which proves the lemma. □

Extension to sequences. Speculative decoding processes the drafted block $\tilde{y}_1, \ldots, \tilde{y}_k$ one position at a time. Writing $y_{1:j-1}$ for the already-committed prefix, at position j the procedure applies exactly the single-token rule to

$$p_j(\cdot) \;=\; p(\cdot \mid x, y_{1:j-1}), \qquad q_j(\cdot) \;=\; q(\cdot \mid x, y_{1:j-1}).$$

(When $\tilde{y}_j$ is rejected, sampling the resample from $(p_j - q_j)_+/Z_j$ terminates the block at position j, which is the same as executing the single-token rule and then stopping.)

We now prove the target-preservation claim $\pi(y_{1:k} \mid x) = p(y_{1:k} \mid x)$ by induction on k.

Base case ($k = 1$): direct from the lemma.

Inductive step: assume $\pi(y_{1:k-1} \mid x) = p(y_{1:k-1} \mid x)$. Conditioned on any particular prefix $y_{1:k-1}$, position k applies the single-token rule to p_k, q_k, so by the lemma

$$\pi(y_k \mid x, y_{1:k-1}) \;=\; p_k(y_k) \;=\; p(y_k \mid x, y_{1:k-1}).$$

Multiplying by the inductive hypothesis,

$$\pi(y_{1:k} \mid x) \;=\; \pi(y_{1:k-1} \mid x) \cdot \pi(y_k \mid x, y_{1:k-1}) \;=\; p(y_{1:k-1} \mid x) \cdot p(y_k \mid x, y_{1:k-1}) \;=\; p(y_{1:k} \mid x).$$

By induction, $\pi(y_{1:k} \mid x) = p(y_{1:k} \mid x)$ for every $k \geq 1$, so the speculative-decoding procedure preserves the target model's distribution exactly. ■

Solution 4. Used Claude (AI assistant by Anthropic) in the following capacities:

- **Solution ideation:** Brainstorming approaches and generating initial ideas for problem-solving strategies, in particular for structuring the transformer truncated-BPTT algorithm, the normalizing-flow construction, and the speculative-decoding correctness proof.
- **English grammar and writing:** Polishing and refining the English writing throughout the solutions to improve clarity, coherence, and academic tone.
- **LaTeX formatting:** Optimizing typesetting and layout of mathematical expressions, pseudo-code, and document structure for better readability and presentation.
- **Result verification:** Cross-checking the correctness of intermediate derivations, the Jacobian determinant computation in Problem 2, and the case-split identity used in Problem 3.

All final solutions were reviewed and verified by the author.